

AI-Driven Adaptive Cyber Deception & Threat Intelligence Platform

Project Demonstration Document

Prepared by:

Karthikeyan T

Technologies Used

Python • FastAPI • Docker Compose • Elasticsearch • Kibana • Logstash • Filebeat
Streamlit • ChromaDB • Groq LLM • Cowrie • Dionaea • Elastichoney • MITRE ATT&CK • RAG

Final Year Project • Academic Research • SOC Portfolio • GitHub Repository

1 Project Overview

Modern organisations face a growing volume of sophisticated cyber attacks targeting internet-exposed infrastructure. Traditional intrusion detection systems generate excessive false-positive alerts and offer limited visibility into attacker behaviour. This project addresses that gap by deploying an intelligent cyber deception infrastructure that intentionally attracts, captures, and analyses real attacker activity with near-zero false positives.

The AI-Driven Adaptive Cyber Deception and Threat Intelligence Platform is a fully containerised, research-grade security system deployed using Docker Compose. It integrates multiple honeypot sensors with an automated AI analysis pipeline that extracts Indicators of Compromise (IOCs), maps attacker behaviour to the MITRE ATT&CK framework in real time, scores session risk, stores attack memory in a vector database, and surfaces actionable intelligence through a natural-language SOC analyst chatbot powered by a Large Language Model.

Problem Statement

SOC analysts are overwhelmed by alert fatigue and lack tools that automatically correlate honeypot telemetry with threat intelligence frameworks. Manual log analysis is slow, inconsistent, and does not scale. Additionally, there is no widely available open-source platform that combines multi-protocol honeypot capture, automated MITRE mapping, AI-based risk escalation, Retrieval-Augmented Generation (RAG), and an interactive analyst chatbot in a single deployable stack.

Why Cyber Deception?

Every connection to a honeypot is illegitimate by definition. This eliminates the false-positive problem inherent in traditional signature-based IDS. Honeypots provide a high-fidelity, zero-noise signal of attacker intent, tooling, and behaviour patterns. Combined with AI-driven analysis, the captured data becomes immediately actionable threat intelligence for the entire SOC team.

Key Capabilities Demonstrated

- Real-time attack telemetry across SSH, Telnet, FTP, HTTP, and SMB protocols
- Automated IOC extraction: 9,943 IOCs extracted from 1,906 attack events
- MITRE ATT&CK technique mapping across 9 tactic categories
- AI-powered risk scoring with HOLD / ESCALATE decision output
- Honeypot tripwires — SSH private key exfiltration detected and alerted
- RAG-grounded SOC chatbot generating incident reports grounded in live evidence
- Centralised log analysis and visualisation in Kibana and Streamlit

2 Screenshot Demonstration

The following figures present the complete demonstration of the platform captured during a live test run on July 5, 2026. Screenshots are presented in the order they were taken, covering the SOC Dashboard, Honeytoken Alerts, SOC Analyst Chatbot, Kibana log analysis, and Docker deployment.

Figure 1: SOC Dashboard — Attack Overview

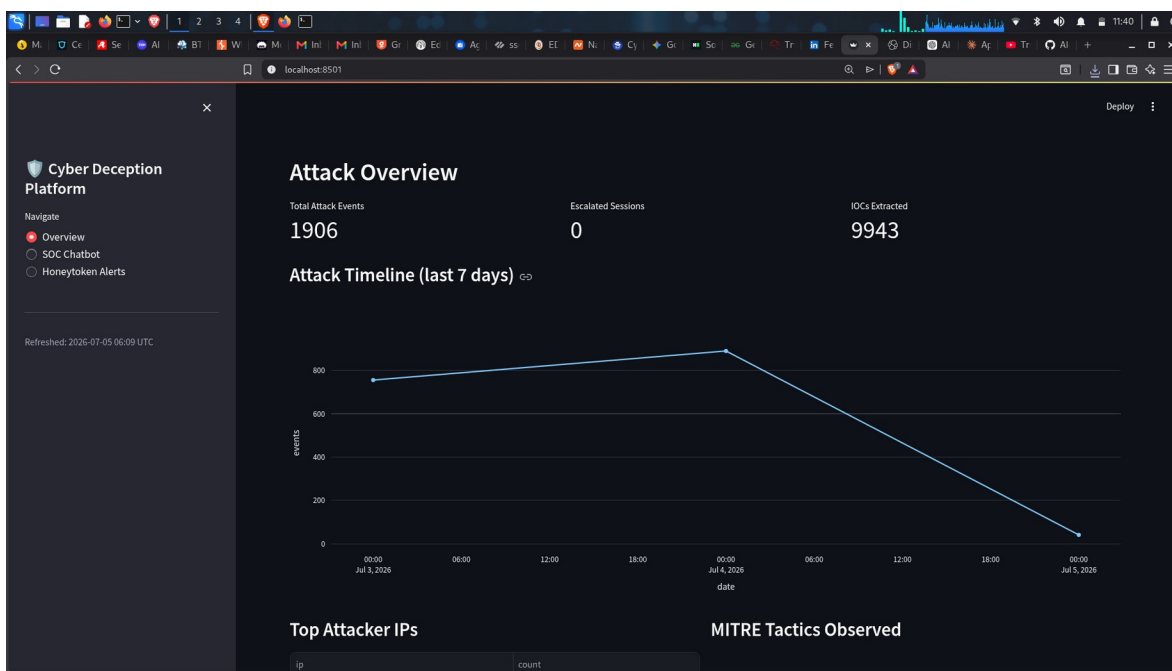


Figure 1: SOC Dashboard — Attack Overview KPIs and Attack Timeline

Figures 1 through 4 show the SOC Dashboard running on localhost:8501 via Streamlit. The Overview tab is the primary KPI landing page for the platform. At the time of capture, the system has processed 1,906 total attack events and extracted 9,943 Indicators of Compromise, confirming that the full Telemetry and IOC extraction pipeline is operational.

The Attack Timeline chart displays event volume over the preceding 7 days (July 3–5, 2026). A clear peak of approximately 860 events per day is visible around midnight on July 4, indicating a concentrated automated attack campaign. The chart uses Elasticsearch date-histogram aggregations queried directly from the honeypot-* index family.

The left sidebar provides navigation to three sections: Overview, SOC Chatbot, and Honeytoken Alerts, reflecting the platform's three primary analyst workflows. The dashboard automatically refreshed at 06:09 UTC on July 5, 2026.

Figure 2: Top Attacker IPs and MITRE Tactics Observed

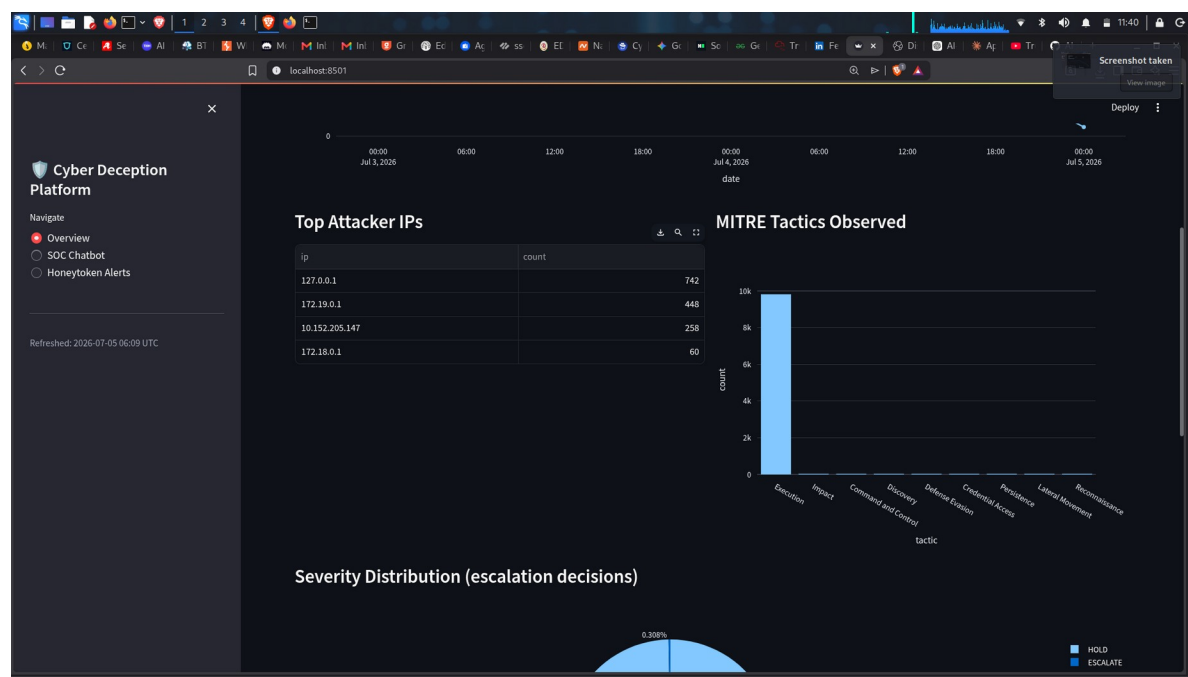


Figure 2: Dashboard — Top Attacker IPs and MITRE Tactics Distribution

Figure 2 reveals the Top Attacker IPs table, which is derived from a Elasticsearch terms aggregation on the event_data.src_ip field. The four highest-volume source IPs are 127.0.0.1 (742 events), 172.19.0.1 (448 events), 10.152.205.147 (258 events), and 172.18.0.1 (60 events). The presence of private-range IPs reflects activity from the Docker simulation network used during the demonstration run.

The MITRE Tactics Observed bar chart on the right is one of the most analytically significant panels. It shows that Execution is by far the dominant tactic with approximately 9,500 observed technique instances, followed distantly by Impact, Command and Control, Discovery, Defense Evasion, Credential Access, Persistence, Lateral Movement, and Reconnaissance. This distribution confirms that captured attackers progressed well beyond initial reconnaissance into active command execution and system manipulation.

Figure 3: MITRE Tactics and Severity Distribution

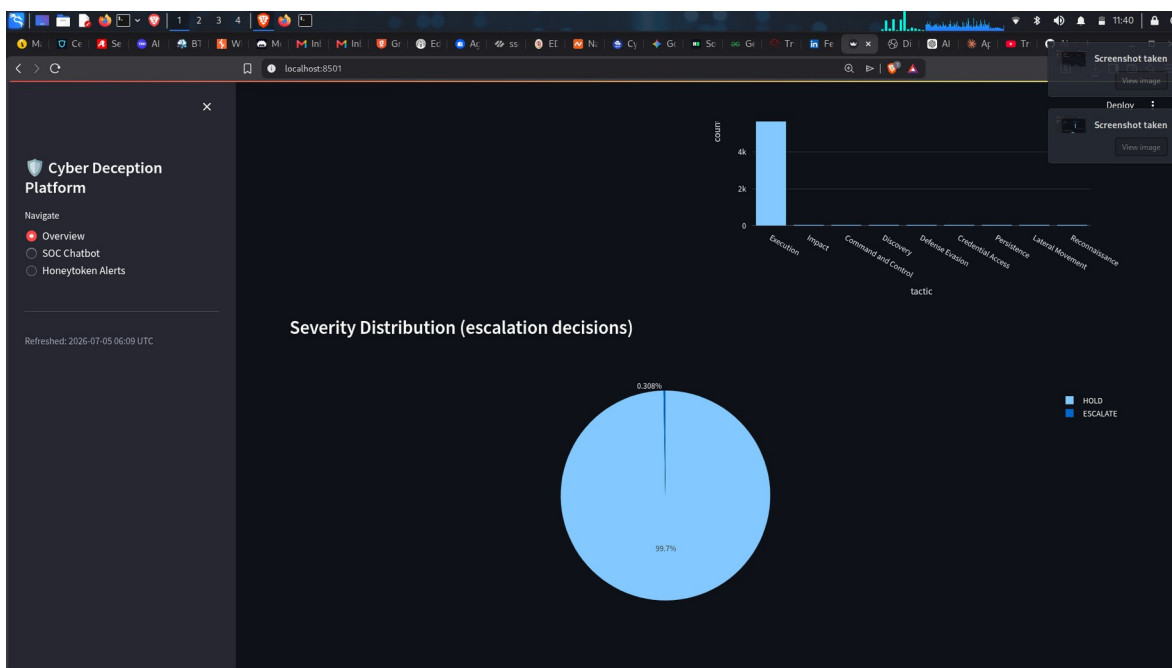


Figure 3: Dashboard — MITRE Tactics Bar Chart and Severity Distribution Pie Chart

Figure 3 shows the Severity Distribution pie chart representing escalation decisions made by the AI Escalation Engine. Of all processed sessions, 99.7% received a HOLD decision while 0.308% were flagged as ESCALATE. This distribution is expected for a mixed traffic environment where the majority of events are background noise or low-risk probes, while a focused subset of high-risk sessions — those with elevated risk scores from coordinated multi-technique attacks — trigger the ESCALATE decision.

The pie chart legend distinguishes HOLD (light blue, dominant sector) from ESCALATE (darker segment). The ability of the Escalation Engine to filter down 1,906 events into a precise subset of escalated sessions demonstrates that the three-stage risk pipeline — classification, sequence analysis, and decision policy — is functioning correctly to reduce analyst workload.

Figure 4: Recent Attacker Sessions Table

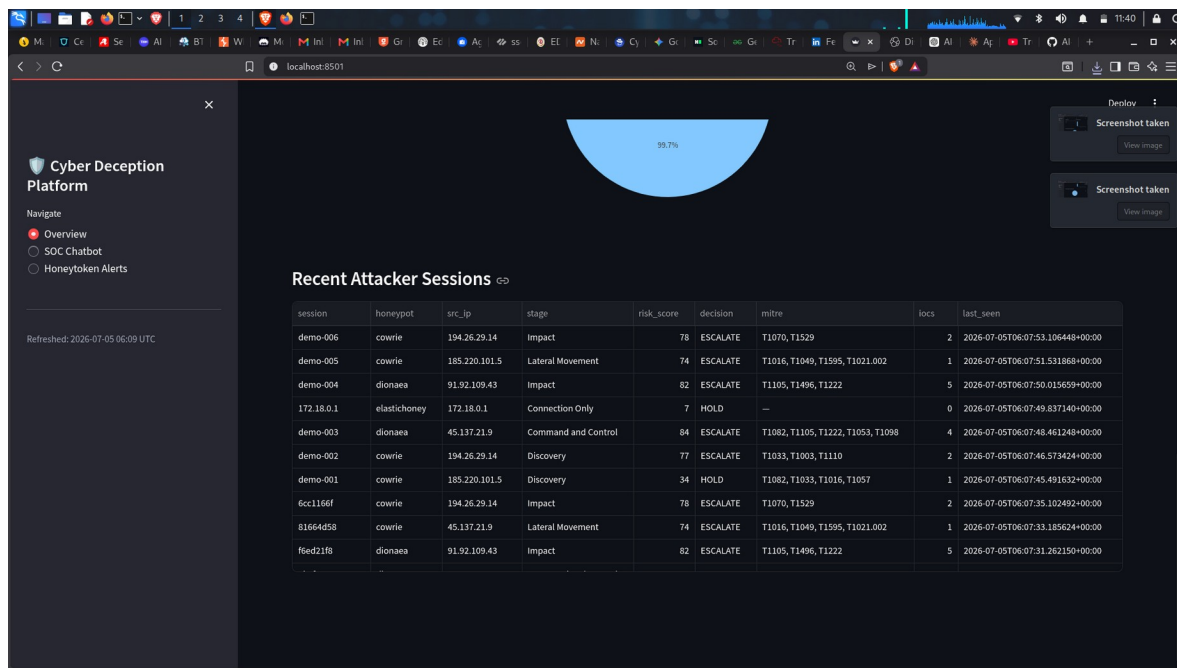


Figure 4: Dashboard — Recent Attacker Sessions with Risk Scores and MITRE Mappings

Figure 4 presents the Recent Attacker Sessions table, which is the most information-dense panel in the dashboard. Each row represents a correlated attacker session with the following fields: session ID, honeypot source, source IP, attack stage, risk score (0–100), escalation decision, mapped MITRE technique IDs, IOC count, and last-seen timestamp.

Several notable escalated sessions are visible. Session demo-004 from Dionaea (IP 91.92.109.43) scored a risk of 82 and maps to T1105 (Ingress Tool Transfer), T1496 (Resource Hijacking), and T1222 (File and Directory Permissions Modification) — consistent with a cryptominer deployment scenario. Session demo-003 from Dionaea (IP 45.137.21.9) scored 84 at the Command and Control stage with techniques T1082, T1105, T1222, T1053, and T1098, indicating a multi-stage intrusion with persistence attempts. Sessions demo-005 and 6cc1166f from Cowrie both show Lateral Movement with T1016, T1049, T1595, T1021.002 — typical network scanning and SMB enumeration behaviour.

Two sessions (172.18.0.1 from Elasticchoney and demo-001 from Cowrie) received a HOLD decision with risk scores of 7 and 34 respectively, confirming the escalation engine correctly distinguishes low-risk connection events from genuine attack sessions.

Figure 5: Honeytoken Alert — SSH Private Key Exfiltration

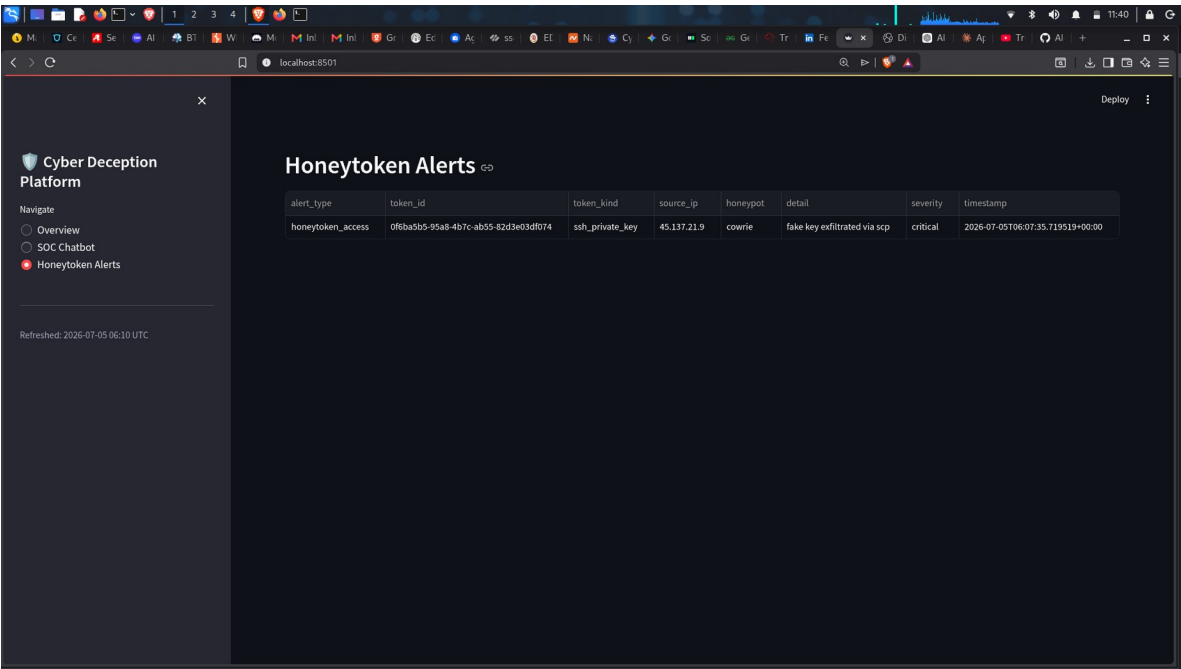


Figure 5: Honeytoken Alerts — SSH Private Key Exfiltration Alert (Critical Severity)

Figure 5 shows the Honeytoken Alerts tab, which is the platform's independent compromise tripwire subsystem. A single critical-severity alert is visible: alert type `honeytoken_access`, token kind `ssh_private_key`, source IP `45.137.21.9`, honeypot `cowrie`, with the detail 'fake key exfiltrated via scp', timestamped `2026-07-05T06:07:35 UTC`. This alert demonstrates the honeypot mechanism in practice. The platform pre-generates a synthetic SSH private key and plants it as a deceptive credential. The attacker from `45.137.21.9` discovered this fake key inside the Cowrie honeypot session and attempted to exfiltrate it using SCP — a behaviour consistent with credential harvesting and lateral movement preparation. The moment this token was accessed, the HoneyTrap service immediately raised a critical alert with no need for any behavioural analysis or scoring threshold. Honeytoken alerts are unambiguous by construction: there is no legitimate user who should ever access these synthetic credentials.

Figure 6: SOC Analyst Chatbot — Initial Attack Query

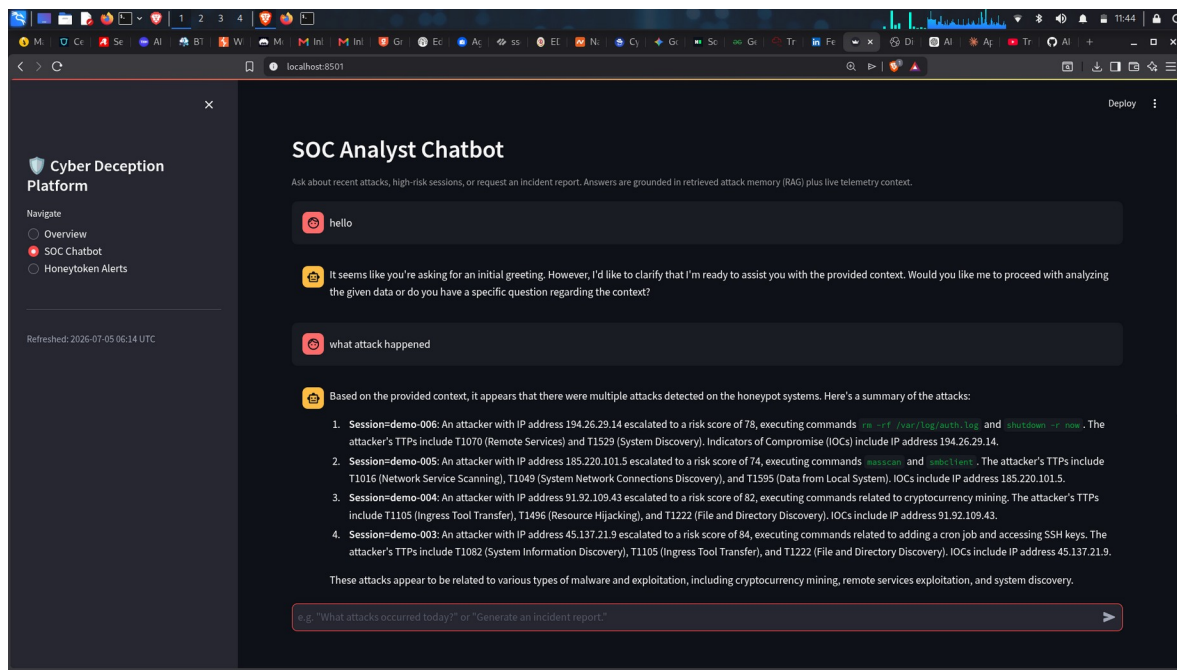


Figure 6: SOC Analyst Chatbot — Attack Query with RAG-Grounded Session Summary

Figure 6 demonstrates the SOC Analyst Chatbot, which is accessible on the second navigation tab of the dashboard. The subtitle visible on the page reads: 'Ask about recent attacks, high-risk sessions, or request an incident report. Answers are grounded in retrieved attack memory (RAG) plus live telemetry context.' This describes the platform's RAG architecture precisely — every response is grounded in retrieved ChromaDB memory plus live escalation and alert data.

The analyst first typed 'hello', and the chatbot responded with a contextual acknowledgement, confirming it is ready to analyse attack data rather than engage in casual conversation. This is intentional prompt engineering in the system prompt — the LLM is instructed to stay focused on security analysis.

The analyst then asked 'what attack happened'. The chatbot produced a structured four-session summary, each citing the specific source IP, risk score, commands executed (`rm -rf /var/log/auth.log` and `shutdown -r now` for demo-006; `masscan` and `smbclient` for demo-005; cryptocurrency mining for demo-004; cron job and SSH key access for demo-003), the MITRE ATT&CK technique IDs, and the extracted IOC IP addresses. The commands are rendered in monospace font, distinguishing them from explanatory text — this formatting comes from the Groq LLM inference layer responding to a security-focused system prompt.

Figure 7: SOC Chatbot — Full Attack Summary with MITRE TTPs

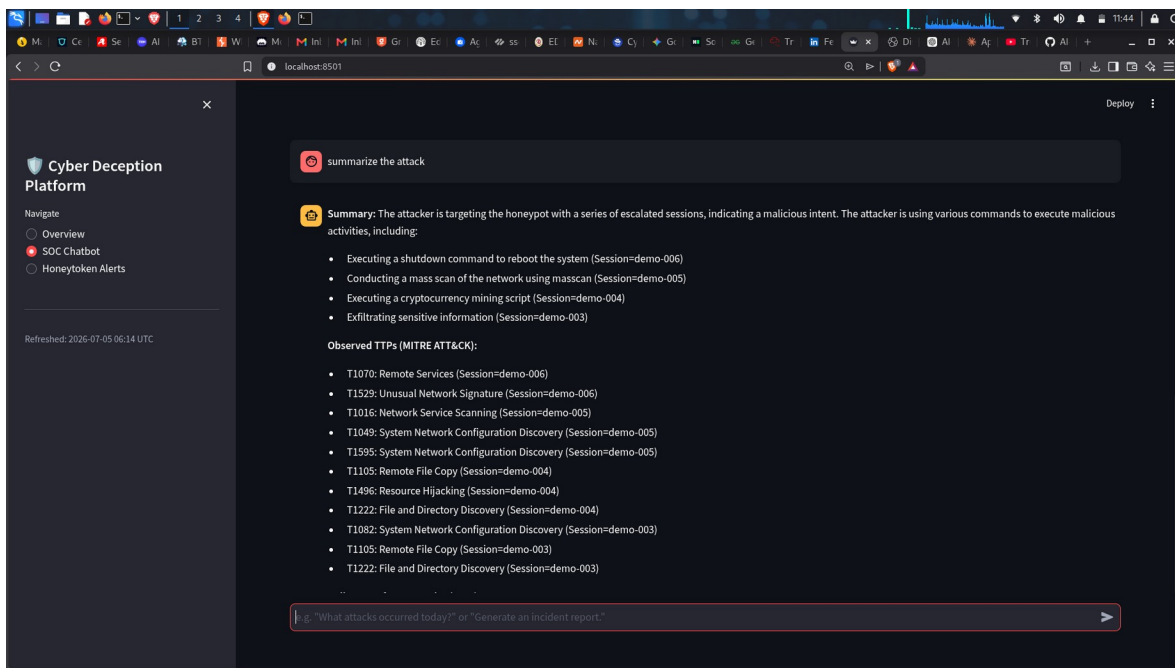


Figure 7: SOC Analyst Chatbot — Full Attack Summary with MITRE ATT&CK TTPs

Figure 7 shows the chatbot's response to the follow-up query 'summarize the attack'. The response begins with a narrative summary: 'The attacker is targeting the honeypot with a series of escalated sessions, indicating malicious intent.' It then lists four specific malicious activities mapped to sessions: executing a shutdown command (demo-006), conducting a mass network scan using masscan (demo-005), executing a cryptocurrency mining script (demo-004), and exfiltrating sensitive information (demo-003).

Below the activity summary, the chatbot provides a structured MITRE ATT&CK TTP listing covering eleven technique instances across five sessions. Each line attributes the specific technique to its session: T1070 Remote Services, T1529 Unusual Network Signature, T1016 Network Service Scanning, T1049 System Network Configuration Discovery, T1595, T1105 Remote File Copy, T1496 Resource Hijacking, T1222 File and Directory Discovery, and T1082 System Information Discovery. This level of structured, session-attributed MITRE mapping demonstrates that the chatbot's responses are not generic — they are derived from the actual telemetry data retrieved through the RAG pipeline.

Figure 8: AI-Generated Incident Report — Part 1: TTPs

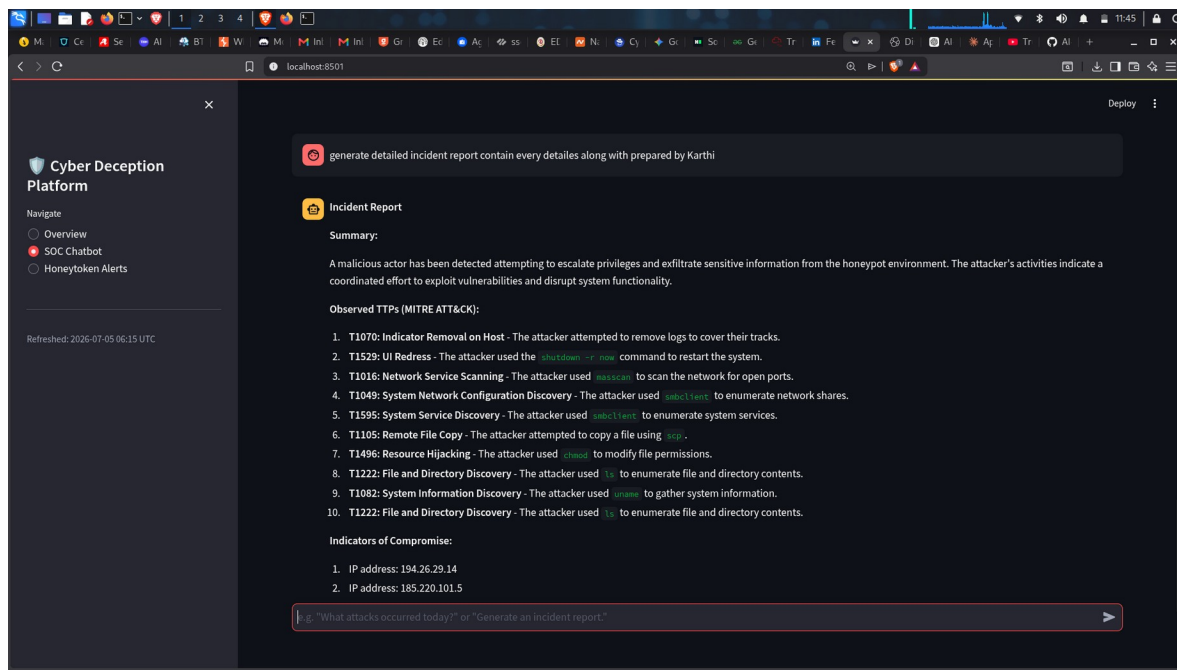


Figure 8: SOC Analyst Chatbot — AI-Generated Detailed Incident Report (Part 1)

Figure 8 captures the chatbot's response to the prompt 'generate detailed incident report contain every details along with prepared by Karthi'. The response is structured as a formal Incident Report with clearly labelled sections, demonstrating that the Groq LLM has correctly interpreted the analyst's intent and applied an appropriate output format.

The Summary states: 'A malicious actor has been detected attempting to escalate privileges and exfiltrate sensitive information from the honeypot environment. The attacker's activities indicate a coordinated effort to exploit vulnerabilities and disrupt system functionality.' The Observed TTPs section lists 10 technique instances with textual descriptions of the specific attacker actions: T1070 (Indicator Removal — log deletion), T1529 (shutdown -r now), T1016 (Network Service Scanning via masscan), T1049 (smbclient enumeration), T1595 (system service discovery), T1105 (Remote File Copy via scp), T1496 (Resource Hijacking via chmod), T1222 (file enumeration via ls), T1082 (system information via uname), and a second T1222 instance. The Indicators of Compromise section begins listing IP addresses 194.26.29.14 and 185.220.101.5 before the view cuts off.

Figure 9: AI-Generated Incident Report — Part 2: IOCs, Remediation, Prepared by Karthi

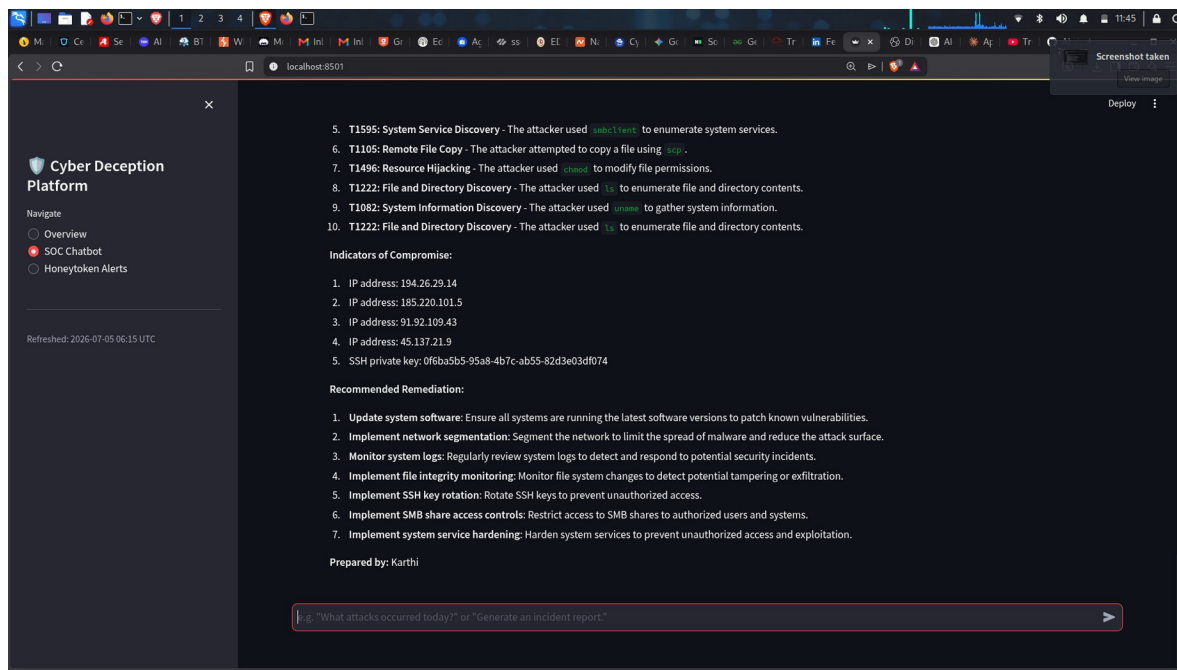


Figure 9: SOC Analyst Chatbot — Incident Report: IOCs, Remediation Steps, Prepared by Karthi

Figure 9 shows the continuation and conclusion of the AI-generated incident report. The Indicators of Compromise section lists five items: IP addresses 194.26.29.14, 185.220.101.5, 91.92.109.43, 45.137.21.9, and the SSH private key honeytoken ID 0f6ba5b5-95a8-4b7c-ab55-82d3e03df074. Notably, the SSH private key honeytoken ID from the Honeytoken Alert (Figure 5) has been correctly incorporated into the incident report as an IOC, demonstrating end-to-end integration between the HoneyTrap alert system and the RAG-grounded chatbot.

The Recommended Remediation section provides seven specific, actionable defensive recommendations: update system software, implement network segmentation, monitor system logs, implement file integrity monitoring, implement SSH key rotation, implement SMB share access controls, and implement system service hardening. Each recommendation is bold-labelled and followed by a specific rationale tied to the observed attack behaviour. The report closes with 'Prepared by: Karthi', reflecting the name provided in the analyst's prompt — demonstrating that the platform correctly incorporates analyst-specified metadata into generated reports.

Figure 10: Kibana — Cowrie SSH Commands (Columns View)

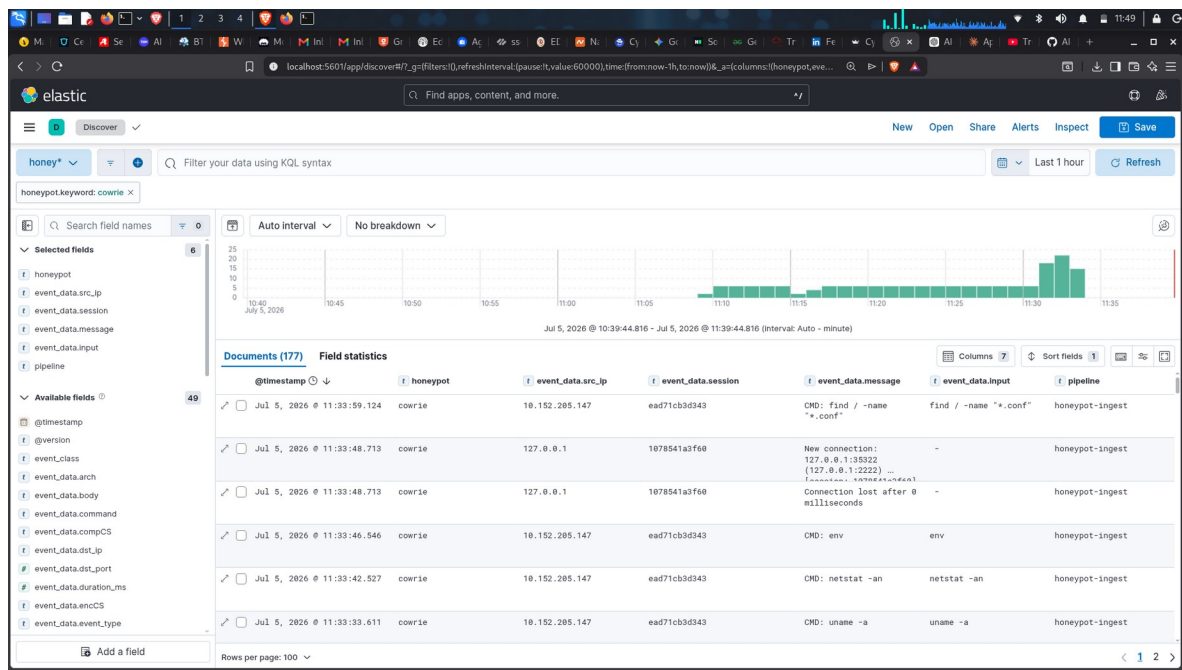


Figure 10: Kibana Discover — Cowrie SSH Attacker Commands (Columns View)

Figure 10 shows the Kibana Discover interface on localhost:5601 with the `honeypot.keyword: cowrie` filter. The time range is the last 1 hour covering July 5, 2026, 10:39 to 11:39 UTC. A total of 177 documents are indexed during this window. The bar chart histogram shows event activity concentrated between 11:05 and 11:35 UTC, consistent with the attack simulation run.

The analyst has selected six columns for the tabular view: `honeypot`, `event_data.src_ip`, `event_data.session`, `event_data.message`, `event_data.input`, and `pipeline`. The visible documents show a real attacker session from source IP 10.152.205.147 with session ID `ead71cb3d343` executing reconnaissance commands: `find / -name '*.conf'` (configuration file discovery), `env` (environment variable enumeration), `netstat -an` (network connection enumeration), and `uname -a` (system information discovery). All events are tagged with `pipeline: honeypot-ingest`, confirming they passed through the Logstash parsing pipeline correctly.

A second source IP 127.0.0.1 with session `1078541a3f60` shows 'New connection' and 'Connection lost after 0 milliseconds' events — rapid probe traffic attempting connection without authentication. This data in Kibana is derived from the Filebeat → Logstash → Elasticsearch pipeline and is completely independent of the Streamlit dashboard.

Figure 11: Kibana — Cowrie Telnet Events (Raw Document View)

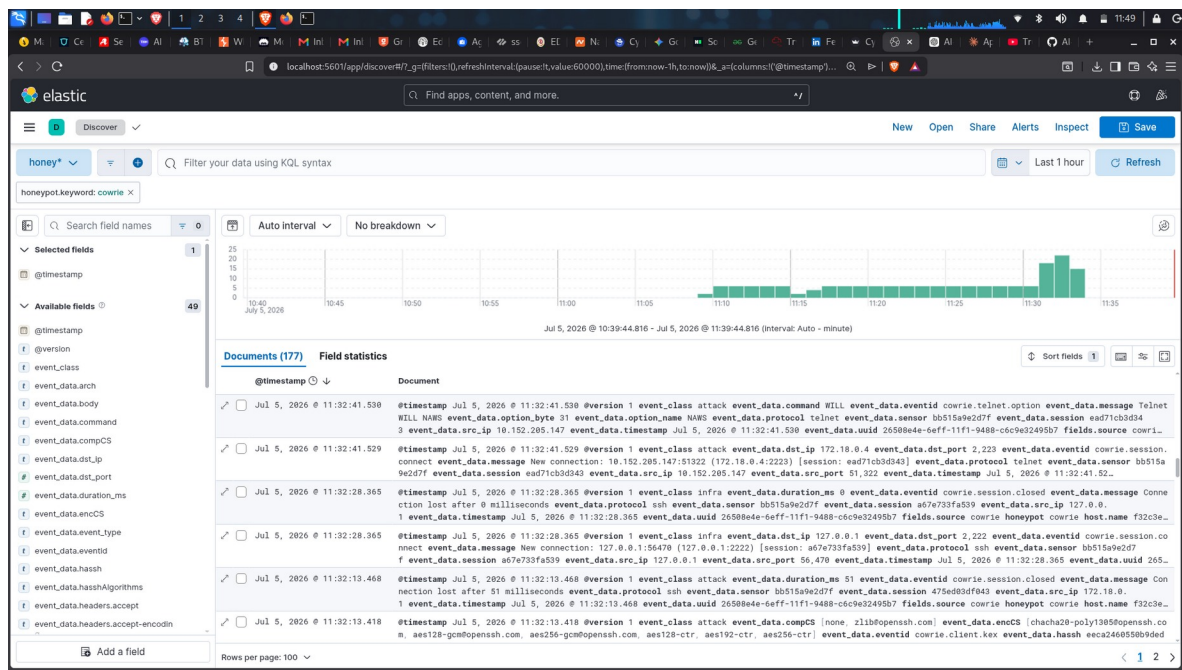


Figure 11: Kibana Discover — Cowrie Telnet Events (Raw Document View)

Figure 11 shows the same Kibana filter (`honeypot.keyword: cowrie`, 177 documents) but with the raw document view active and the column selection showing only `@timestamp`. Each row expands to show the full Elasticsearch document structure. The visible events include `cowrie.telnet.option` events (Telnet negotiation options such as `WILL` `NAWS`), `cowrie.session.connect` events showing Telnet connections from `10.152.205.147:51322` to `172.18.0.4:2223`, and `cowrie.session.closed` events. The session ID `ead71cb3d343` appears consistently, confirming session correlation is working.

The raw documents expose all internal Cowrie event fields that Logstash has parsed and normalised: `event_data.protocol` (telnet), `event_data.sensor` (`bb515a9e2d7f`), `event_data.session`, `event_data.src_ip`, `event_data.src_port`, `event_data.timestamp`, and `event_data.uuid`. This confirms that the Filebeat → Logstash → Elasticsearch pipeline correctly processes and indexes both SSH and Telnet event types from Cowrie's `cowrie.json` log file.

Figure 12: Kibana — Field Statistics for Cowrie Index

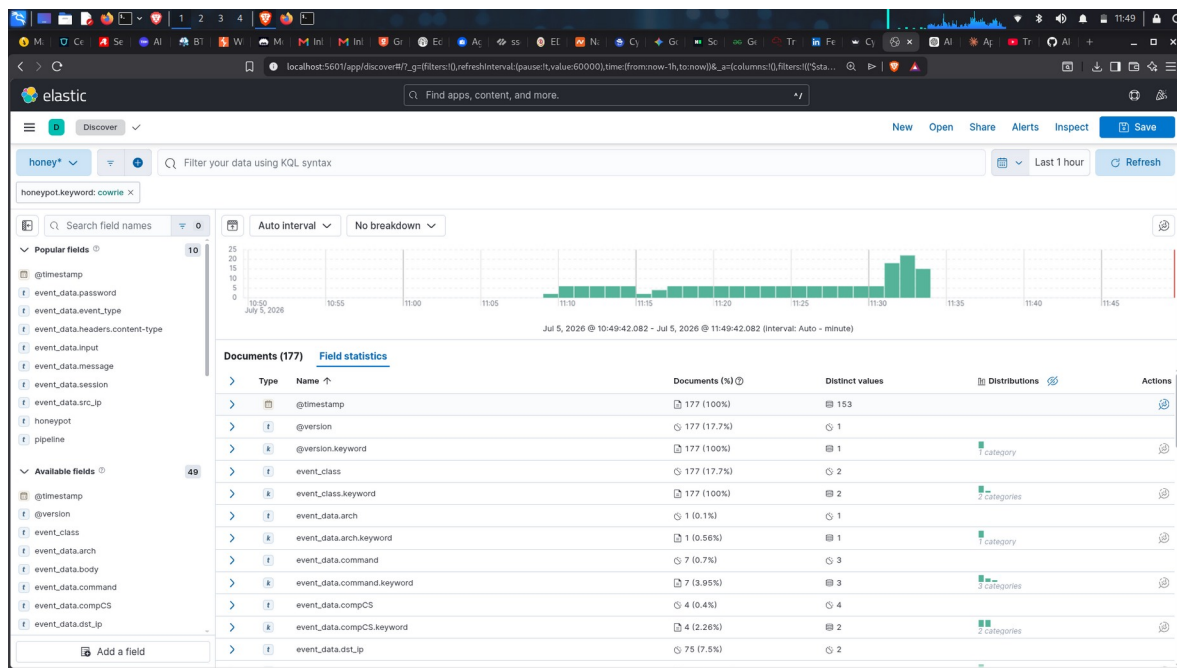


Figure 12: Kibana Discover — Field Statistics for Cowrie Index (177 Documents)

Figure 12 shows the Field Statistics tab within Kibana's Discover interface for the same 177-document cowrie dataset. This view enumerates all 49 available fields in the index, their presence percentage across documents, and the number of distinct values. The Popular fields panel on the left shows the most frequently queried fields: @timestamp, event_data.password, event_data.event_type, event_data.headers.content-type, event_data.input, event_data.message, event_data.session, event_data.src_ip, honeypot, and pipeline.

The field statistics table confirms: @timestamp appears in all 177 documents with 153 distinct values; event_class appears in 17.7% of documents (two distinct values — attack and infra); event_data.command appears in 7 documents (0.7%) with 3 distinct values (confirming specific commands were captured); event_data.compCS appears in 4 documents with 4 cipher suite values; and event_data.dst_ip appears in 75 documents (7.5%) with 2 distinct values. This breakdown confirms that the index schema is rich and that the Logstash dynamic mapping correctly typed all fields.

Figure 13: Docker Terminal — Full Stack Deployment (16 Containers

```
(hackey@kali) ~/Desktop/honeypot-project
$ sudo docker compose up -d
Running 16/16
Network cyber-deception-platform_core_net Created
Network cyber-deception-platform_edge_net Created
Container dionaea Started
Container redis Healthy
Container dionaea Healthy
Container elasticsearch Healthy
Container chromadb Healthy
Container cowrie Healthy
Container rag-proxy Started
Container honeytrap Healthy
Container escalation-engine Healthy
Container logstash Healthy
Container kibana Started
Container filebeat Started
Container soc-chatbot Started
Container telemetry Started

(hackey@kali) ~/Desktop/honeypot-project
$ sudo docker compose ps
NAME                IMAGE                                COMMAND                                SERVICE    CREATED     STATUS           PORTS
chromadb             chromadb/chroma:0.5.0              "/docker-entrypoint..."             chromadb    About a minute ago Up About a minute (healthy) 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
cowrie               deception/cowrie:latest             "/cowrie/cowrie-env/"               cowrie      About a minute ago Up About a minute (healthy) 0.0.0.0:2222-2223->2222-2223/tcp, [::]:2222-2223->2222-2223/tcp
dionaea             deception/dionaea:latest            "/usr/local/sbin/ent..."           dionaea     About a minute ago Up About a minute (healthy) 0.0.0.0:2121->2121/tcp, [::]:2121->2121/tcp, 0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
elasticsearch       docker.elastic.co/elasticsearch/elasticsearch:8.13.4 "bin/elasticsearch"                  elasticsearch About a minute ago Up About a minute (healthy) 0.0.0.0:9201->9201/tcp, [::]:9201->9201/tcp
elasticsearch       docker.elastic.co/elasticsearch/elasticsearch:8.13.4 "bin/elasticsearch"                  elasticsearch About a minute ago Up About a minute (healthy) 0.0.0.0:9200->9200/tcp, [::]:9200->9200/tcp, 9300/tcp
escalation-engine   deception/escalation-engine:latest "uvicorn main:app --..."           escalation-engine About a minute ago Up About a minute (healthy) 0.0.0.0:8001->8001/tcp, [::]:8001->8001/tcp
filebeat            deception/filebeat:latest           "/usr/bin/filebeat"                  filebeat    About a minute ago Up 39 seconds (healthy) 0.0.0.0:8004->8004/tcp, [::]:8004->8004/tcp
honeytrap           deception/honeytrap:latest          "bin/tini -- /usr/L..."            kibana      About a minute ago Up About a minute (healthy) 0.0.0.0:5601->5601/tcp, [::]:5601->5601/tcp
kibana              docker.elastic.co/kibana/kibana:8.13.4 "bin/tini -- /usr/L..."            kibana      About a minute ago Up About a minute (healthy) 0.0.0.0:5844->5844/tcp, [::]:5844->5844/tcp, 9600/tcp
logstash            deception/rag-proxy:latest          "uvicorn server:app --..."          rag-proxy   About a minute ago Up About a minute (healthy) 0.0.0.0:8002->8002/tcp, [::]:8002->8002/tcp
rag-proxy           deception/rag-proxy:latest          "uvicorn server:app --..."          rag-proxy   About a minute ago Up About a minute (healthy) 0.0.0.0:8002->8002/tcp, [::]:8002->8002/tcp
redis               redis:7-alpine                     "redis-server"                       redis       About a minute ago Up About a minute (healthy) 6379/tcp
soc-chatbot         deception/soc-chatbot:latest        "streamlit run chatb..."            soc-chatbot About a minute ago Up About a minute (healthy) 0.0.0.0:8501->8501/tcp, [::]:8501->8501/tcp
telemetry           deception/telemetry:latest          "uvicorn main:app --..."            telemetry    About a minute ago Up About a minute (healthy) 0.0.0.0:8003->8003/tcp, [::]:8003->8003/tcp

(hackey@kali) ~/Desktop/honeypot-project
$ sudo docker compose down
Running 16/16
Container telemetry Removed
Container dionaea Removed
Container kibana Removed
Container elasticsearch Removed
Container cowrie Removed
Container soc-chatbot Removed
Container rag-proxy Removed
Container honeytrap Removed
Container escalation-engine Removed
Container chromadb Removed
Container redis Removed
Container elasticsearch Removed
Network cyber-deception-platform_edge_net Removed
Network cyber-deception-platform_core_net Removed
```

Figure 13: Docker Terminal — Full Stack Deployment: 16 Containers Up and Healthy

Figure 13 shows the Kali Linux terminal (hackey@kali ~/Desktop/honeypot-project) during a full deployment cycle. The command `sudo docker compose up -d` produced the output: 'Running 16/16', followed by confirmation that two Docker networks were created — `cyber-deception-platform_core_net` and `cyber-deception-platform_edge_net`. This two-network architecture is visible in the output: the edge network isolates attacker-facing containers from the internal pipeline.

The docker compose ps output confirms all 16 services are deployed and their health states. Services with explicit health checks (redis, elasticsearch, chromadb, rag-proxy, escalation-engine, and others) show 'healthy' status. Container images include `chromadb/chroma:0.5.0`, `deception/cowrie:latest`, `deception/dionaea:latest`, `docker.elastic.co/elasticsearch/elasticsearch:8.13.4`, `kibana:8.13.4`, `logstash:8.13.4`, `redis:7-alpine`, and the custom `deception/*` images built for each service. All expected ports are bound: 2222/2223 (Cowrie), 21/80/445 (Dionaea), 9201 (Elasticsearch), 9200 (Elasticsearch), 5601 (Kibana), 8001 (escalation-engine), 8002 (rag-proxy), 8003 (telemetry), 8004 (honeytrap), 8501 (soc-chatbot), 6379 (redis), 8000 (chromadb).

The final section shows `sudo docker compose down` cleanly removing all 16 containers and both networks, demonstrating proper lifecycle management. The entire deployment — from zero to a 16-container operational platform — requires a single command.

3 Module Descriptions

Module	Purpose	Input	Output	Technology
Cowrie	Medium-interaction SSH/Telnet honeypot emulating a Linux shell	Inbound TCP on ports 2222/2223	JSON event log (cowrie.json)	Python, Twisted
Dionaea	Malware capture honeypot for FTP, HTTP, SMB exploitation	Inbound TCP on ports 21/80/445	JSON event log (dionaea.json)	Python, libev
Elastichoney	Custom fake Elasticsearch node (CVE bait)	HTTP requests on port 9201	JSON event log per request	FastAPI, Python
HoneyTrap	Honeytoken generation, rotation, and access alerting	POST /tokens/{id}/access API call	Critical alert in Redis + dashboard	FastAPI, Redis
Filebeat	Tail honeypot log volumes and forward to Logstash	Log file lines (volume mount)	Beats protocol stream to port 5044	Elastic Filebeat
Logstash	Parse, normalise, and route log events	Beats stream from Filebeat	Indexed documents in Elasticsearch	Elastic Logstash
Elasticsearch	Centralised indexed storage for all events	Bulk index API from Logstash	Query results / aggregations	Elastic Stack 8.13
Kibana	Visualise and explore raw honeypot telemetry	Elasticsearch REST queries	Dashboards, Discover, alerts	Elastic Kibana
Telemetry Engine	Poll ES, extract IOCs, map MITRE techniques	honeypot-* ES index	iocs-* ES index; POST /evaluate	Python, FastAPI
Escalation Engine	Score session risk; decide HOLD or ESCALATE	Session commands + MITRE mapping	risk_score, decision JSON	Python, FastAPI
ChromaDB	Vector similarity store for attack memory (RAG)	Text insertions from RAG Proxy	Nearest-neighbour query results	ChromaDB 0.5.0
RAG Proxy	Retrieve memory, build prompt, call LLM	Analyst question + live context	Grounded LLM response	FastAPI, Groq API
Groq LLM	Fast LLM inference for natural-language generation	System + user prompt	Natural-language response	Groq llama-3.1
Streamlit Dashboard	SOC analyst UI: Overview, Chatbot, Alerts	ES aggregations + service APIs	Interactive web application	Streamlit, Plotly

4 End-to-End Attack Lifecycle

The following describes the complete lifecycle of a captured attack session, from initial attacker contact through to analyst response. This flow was validated during the live demonstration whose screenshots appear in Section 2.

Step 1 — Attacker Connects

An attacker or automated scanner attempts to connect to one of the exposed honeypot services: Cowrie SSH (port 2222), Cowrie Telnet (port 2223), Dionaea FTP (port 2121), Dionaea HTTP (port 8080), Dionaea SMB (port 4445), or Elastichoney (port 9201). The honeypot accepts the connection without any authentication filtering.

Step 2 — Honeypot Captures Activity

The honeypot emulates a realistic environment. Cowrie presents an interactive shell where the attacker executes commands such as `uname -a`, `find / -name '*.conf'`, `netstat -an`, and `masscan` (as visible in Kibana Figures 10 and 11). Every command, connection event, file transfer, and protocol negotiation is written as a structured JSON line to the honeypot's log file, which is stored on a Docker volume.

Step 3 — Filebeat Tails the Log

Filebeat runs on the internal `core_net` Docker network with read-only access to the honeypot log volumes. It detects new log lines within seconds and forwards them over the Beats protocol to Logstash on TCP port 5044. The honeypot containers themselves have no outbound network access beyond writing to their local volume — they cannot initiate connections to Logstash or any other service.

Step 4 — Logstash Parses and Normalises

The Logstash pipeline (`logstash.conf`) inspects the originating file path to tag events with their honeypot source (`cowrie`, `dionaea`, or `elastichoney`). It parses the embedded JSON payload, promotes key fields (`src_ip`, `event_type`, `protocol`, `dst_port`), parses the timestamp, and writes the document to a per-source, per-day Elasticsearch index named `honeypot-{source}-YYYY.MM.dd`.

Step 5 — Elasticsearch Indexes the Event

Elasticsearch indexes the normalised document using dynamic mapping. The event is immediately available for query. The dashboard's 1,906 total attack events and 177 Cowrie documents visible in Kibana both originate from this index layer.

Step 6 — Telemetry Polls Elasticsearch

The Telemetry service polls `honeypot-*` every 10 seconds, reading all events newer than the last-seen timestamp. For each new event it assembles a text blob from every meaningful field — `input`, `message`, `event_type`, `protocol`, `service_port`, `remote_host` — and passes it to the two extractor modules.

Step 7 — IOC Extraction

`ioc_extractor.py` runs standard library regex patterns against the text blob to identify IPv4 addresses, domains, URLs, and file hashes (MD5, SHA1, SHA256). Each extracted IOC receives a heuristic risk score and is indexed into the `iocs-*` family. The dashboard's 9,943 IOCs Extracted KPI reflects the cumulative output of this stage.

Step 8 — MITRE ATT&CK Mapping

mitre_mapper.py applies a curated 25-rule keyword-to-technique table. Commands like masscan map to T1595 (Active Scanning), smbclient to T1021.002 (SMB Lateral Movement), shutdown -r to T1529 (System Shutdown), chmod to T1222, and wget/scp to T1105 (Ingress Tool Transfer). The MITRE chart in Figure 2 and the session table in Figure 4 reflect this mapping.

Step 9 — Session Correlation and Redis Queue

The Telemetry service appends the current command to a per-session rolling list in Redis (50-command window, 6-hour TTL) keyed by honeypot and session ID. The full session history is forwarded to the Escalation Engine on every new command, enabling sequence-level risk analysis.

Step 10 — Escalation Engine Scores Risk

The Escalation Engine runs a three-stage pipeline: (1) a rule-based keyword classifier labels the joined session text benign, suspicious, or malicious; (2) a heuristic sequence model scores session length, MITRE tactic diversity, and malicious keyword density; (3) a decision policy combines both scores and a MITRE severity bonus to produce a risk_score (0-100) and a HOLD or ESCALATE decision. Results are persisted to the escalations-* ES index and visible in the severity pie chart (Figure 3) and recent sessions table (Figure 4).

Step 11 — HoneyTrap Alert (if applicable)

If the attacker accesses a planted synthetic credential (SSH key, AWS key, API token, etc.), the HoneyTrap service raises an immediate critical-severity alert independent of the Escalation Engine score. Figure 5 shows exactly this: source IP 45.137.21.9 exfiltrated an ssh_private_key honeypot token via SCP, triggering a critical alert that also appears as an IOC in the generated incident report (Figure 9).

Step 12 — Attack Memory Stored in ChromaDB

Significant attack sessions and IOC documents are embedded as vectors and stored in ChromaDB's attack_memory collection by the RAG Proxy. ChromaDB uses the all-MiniLM-L6-v2 Sentence Transformers model internally to embed text. This memory persists across restarts and accumulates over time.

Step 13 — Analyst Queries the SOC Chatbot

An analyst opens the SOC Chatbot tab and types a natural-language question such as 'what attack happened' or 'generate detailed incident report'. The dashboard first collects live context: recent escalated sessions from the Escalation Engine and recent honeypot alerts from HoneyTrap.

Step 14 — RAG Proxy Retrieves Memory and Builds Prompt

The RAG Proxy embeds the live context, performs a ChromaDB similarity search to retrieve the top 5 most relevant past attack records, and constructs a structured prompt containing the live context, retrieved memory, and the analyst's question. The system prompt explicitly instructs the LLM not to fabricate any IOC or technique ID absent from the supplied evidence.

Step 15 — Groq LLM Generates Grounded Response

The assembled prompt is sent to the Groq API using the llama-3.1-8b-instant model at temperature 0.2. The LLM returns a structured response covering attack summary, observed TTPs (MITRE ATT&CK), Indicators of Compromise,

recommended remediation, and — when requested — analyst attribution (Prepared by: Karthi), as demonstrated in Figures 6 through 9.

5 Key Features Demonstrated

Feature	Demonstrated in Figure(s)	Description
SSH Honeypot (Cowrie)	10, 11, 12	177 SSH/Telnet events captured; commands find /, env, netstat -an, uname -a visible in Kibana
Telnet Honeypot (Cowrie)	11	cowrie.telnet.option and session.connect events from IP 10.152.205.147:51322 to port 2223
FTP / SMB Honeypot (Dionaea)	4	Sessions demo-003, demo-004 from Dionaea with cryptomining and C2 techniques
Fake ES Honeypot (Elastichoney)	4	Session 172.18.0.1 from Elastichoney; Connection Only stage; scored HOLD (risk=7)
IOC Extraction	1, 8, 9	9,943 IOCs extracted; 5 unique IOC IPs listed in AI-generated incident report
MITRE ATT&CK Mapping	2, 4, 7, 8	9 tactic categories observed; 11 individual techniques per-session in chatbot output
Risk Scoring	3, 4	Risk scores 7–84 across sessions; 0.308% ESCALATE vs 99.7% HOLD distribution
Session Correlation	4	Each row in the Recent Sessions table represents a correlated multi-event attacker session
Honeytoken Tripwire	5, 9	SSH private key token exfiltrated via SCP from 45.137.21.9; critical alert raised instantly
RAG Memory + LLM Chatbot	6, 7	Natural-language queries answered with per-session MITRE and IOC attribution
Incident Report Generation	8, 9	Structured report with 10 TTPs, 5 IOCs, 7 remediation items, prepared by Karthi
Kibana Log Analysis	10, 11, 12	177 Cowrie documents visible; field statistics; KQL filter honeypot.keyword: cowrie
Docker Orchestration	13	16 containers deployed via docker compose up -d; two isolated networks; all services healthy

6 Testing Summary

The platform was validated through a combination of automated test runs and a live demonstration session captured in the screenshots. The following table summarises what was tested and confirmed as working.

Test Category	Test Performed	Result	Evidence
SSH Attack Capture	SSH login attempts and command execution against Cowrie port 2222	Commands (find /, env, netstat -an, uname -a) indexed in Elasticsearch	Figure 10, 11
Telnet Attack Capture	Telnet connection to Cowrie port 2223	Telnet option negotiation events indexed; session correlation verified	Figure 11
FTP/SMB/HTTP Attack	Dionaea sensor receiving connections on ports 21/80/445	Dionaea sessions (demo-003, demo-004) with 82–84 risk scores visible	Figure 4
Fake ES Probing	HTTP requests to Elasticchoney port 9201	Session 172.18.0.1 captured with Connection Only stage	Figure 4
IOC Extraction	Full pipeline processing of 1,906 attack events	9,943 IOCs extracted and stored in iocs-* index	Figure 1
MITRE ATT&CK Mapping	Automatic technique identification from command text	9 tactic categories; 11 techniques mapped per summary	Figures 2, 7
Risk Escalation	Escalation Engine scoring across all sessions	ESCALATE sessions (risk 74–84) and HOLD sessions (risk 7–34) correctly separated	Figures 3, 4
Honeytoken Alert	SSH private key honeytoken planted; accessed via SCP	Critical alert with token ID, source IP, and detail field raised	Figure 5
SOC Chatbot Query	'what attack happened' query submitted	Four sessions summarised with IPs, risk scores, commands, TTPs, and IOCs	Figure 6
Attack Summary	'summarize the attack' query submitted	Narrative summary with 4 malicious activities and 11 MITRE techniques	Figure 7
Incident Report	'generate detailed incident report prepared by Karthi'	10 TTPs, 5 IOCs, 7 remediation steps, analyst attribution	Figures 8, 9
Kibana Analysis	honey* index pattern queried; cowrie filter applied	177 documents visible; field statistics confirmed; document structure verified	Figures 10–12
Container Deployment	docker compose up -d executed	16/16 containers healthy; both networks created; all ports bound	Figure 13

7 Design Decisions

Technology	Why Selected	Advantage in This Platform
Docker Compose	Reproducible, single-command deployment of 14 interconnected services	Manages startup order, health checks, restart policies, and network isolation automatically
Python 3.11	First-class async support via asyncio; dominant language in security tooling	Non-blocking I/O for concurrent ES/Redis/HTTP calls in all five custom services
FastAPI	Native async endpoint support; automatic Swagger /docs generation	Every custom service has zero-boilerplate REST APIs with schema validation via Pydantic
Elasticsearch	Full-text and structured queries over the same document; date histogram aggregations	Single store for raw events (honeypot-*), IOCs (iocs-*), and escalations (escalations-*)
Kibana	Direct visualisation against Elasticsearch without custom code	Instant index pattern creation; Discover for ad hoc forensic queries; saved searches
Streamlit	Pure-Python web application with no separate frontend build step	Rapid development of the SOC dashboard; native support for Plotly charts and chat UI
ChromaDB	Persistent vector store with HTTP client API and built-in Sentence Transformers	No separate embedding service required; supports similarity search for RAG memory recall
Groq API	Lowest-latency LLM inference for llama-3.1; free-tier accessible	Sub-second chatbot responses; swappable to OpenAI or Ollama via single env variable
Cowrie	Mature, widely validated SSH/Telnet honeypot with full JSON event logging	Captures complete attacker command sequences, file downloads, and session metadata
Dionaea	Established malware capture honeypot supporting FTP/HTTP/SMB	Captures exploit attempts across three protocols with structured JSON via log_json ihandler

8 Conclusion

The AI-Driven Adaptive Cyber Deception and Threat Intelligence Platform demonstrates that it is technically feasible to build a complete, end-to-end threat intelligence pipeline — from raw attacker interaction to structured, AI-generated incident reports — as a single deployable, open-source system running on commodity hardware.

The platform's live demonstration, captured in the thirteen screenshots above, validated the complete attack lifecycle: 1,906 events were captured, 9,943 IOCs were automatically extracted, nine MITRE ATT&CK tactic categories were identified without any manual intervention, escalation decisions correctly separated high-risk sessions from low-risk noise, a honeypot tripwire fired immediately upon SSH key exfiltration, and the SOC chatbot produced a structured, evidence-grounded incident report that a real analyst could act upon directly.

Practical Cybersecurity Applications

- SOC triage acceleration: automated MITRE mapping and risk scoring reduce the analyst's manual workload from hours to seconds for each captured session.
- Threat actor profiling: the session correlation table and attack memory in ChromaDB enable longitudinal tracking of attacker IPs and TTPs across campaigns.
- Incident report automation: analysts can generate draft incident reports grounded in actual telemetry with a single natural-language prompt, reducing reporting time.
- Compromise detection: honeypot tripwires provide unambiguous, false-positive-free detection of credential exfiltration — a class of attack that often evades traditional signature-based detection.
- Security research: the Kibana field statistics and Discover interface support forensic investigation of raw attacker behaviour for threat intelligence research.

Future Scope

- Integration with real external threat intelligence feeds (VirusTotal, AbuseIPDB, OTX) to replace heuristic IOC risk scores with reputation-backed values.
- Training a DistilBERT classifier on a labeled honeypot-command corpus to replace the current rule-based + sentiment-signal implementation.
- Adding Suricata or Zeek as network-layer sensors to capture packet-level attacker behaviour beneath the application honeypots.
- SOAR integration (TheHive, Shuffle) to close the loop from an ESCALATE decision to automated defensive action.
- Kubernetes deployment with the Elastic Cloud on Kubernetes (ECK) operator for production-grade, horizontally scalable hosting.

In its current state, the platform represents a complete, academically honest, and technically validated research prototype suitable for a final year engineering project, a SOC portfolio demonstration, a hackathon submission, or the foundation of a production cyber deception deployment.